

TNE20003 - Internet and Cybersecurity for
Engineering Application
Final Project Individual Report

Cao Minh Vu
Faculty of Science, Engineering and Technology
Swinburne University of Technology
Hawthorn, Victoria 3122

October 2024

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Report's Objectives	3
2	Project Implementation	3
2.1	Messaging Queueing Telemetry Transport	3
2.2	Pass Task	4
2.3	Credit Task	4
2.4	Distinction Task	5
2.5	High Distinction Task	7

1 Introduction

MQTT (Message Queuing Telemetry Transport) is a lightweight, publish-subscribe protocol designed for constrained devices and low-bandwidth networks. Its simplicity and efficiency make it ideal for IoT applications, where devices often have limited processing power and connectivity. This report will delve into the implementation of an MQTT project at four distinct levels: pass, credit, distinction, and high distinction.

1.1 Project Overview

The Factory Equipment Monitoring and Maintenance System is the project idea I can think of to utilize the MQTT protocol. Equipment failure can result in large output losses and higher maintenance expenses in contemporary industrial plants. An intelligent monitoring system can enhance equipment efficiency overall, optimize maintenance plans, and anticipate possible breakdowns. The system consists of two end devices: an equipment sensors system and a central monitoring system. The first end device measures critical parameters of manufacturing equipment by hardware sensors whereas the second one keeps tracks of sensor readings and publishes alert messages when they surpass the predefined threshold.

1.2 Report's Objectives

The main gist of this report is to demonstrate the purpose of the MQTT protocol in the evaluation of IoT projects, which is the essential exchange of messages between devices with a high focus on flexibility and reliability. Moreover, cybersecurity vulnerabilities will be addressed along with a solution proposal on the current state of the MQTT broker and client usage.

2 Project Implementation

2.1 Messaging Queueing Telemetry Transport

A lightweight publish-subscribe protocol called MQTT (Message Queuing Telemetry Transport) was created for networks with limited capacity and devices with limitations. IBM first created it for use in industrial automation applications in the 1990s.

MQTT is a popular option for Internet of Things (IoT) applications due to its ease of use and effectiveness. It is frequently utilized for:

- **Smart homes:** managing appliances such as security systems, lighting, and thermostats.
- **Industrial IoT:** Production lines, sensors, and machinery monitoring and control.
- **Healthcare:** Remote monitoring, patient data, and medical device tracking.
- **Agriculture:** Keeping an eye on livestock, crops, and environmental factors.
- **Transportation:** Fleet management, route optimization, and vehicle tracking.
- **Smart cities:** Smart cities are those that manage infrastructure, gather data from sensors, and enhance urban services.

MQTT's capacity to manage extensive deployments of devices with constrained resources is the main driving force behind its use in these applications. By ensuring that devices only receive pertinent messages, MQTT's publish-subscribe approach lowers network traffic and boosts efficiency.

2.2 Pass Task

The requirement for the Pass task says that there must be two client devices, one publishes messages to a private topic, and one subscribes to that private topic. The library that I used to write an MQTT client in Python is paho-mqtt. For both clients, I established a connection to the broker server by setting the broker hostname to rule28.i4t.swin.edu.au, setting the port name to 1883, and entering my username and password as <103825154>. After that, the first client published some messages to my private topic "<103825154>/temperature" using the publish() function and printed the dummy sensor value on the terminal. On the subscribe client, I use the same procedure for connection establishment, then I subscribe to the same topic using the subscribe() function. In order to retrieve messages from a subscribe function, I modify the on_message() callback function. By firing the subscribe client code and then firing the publish client code, you can observe that the subscribe client can now retrieve messages that is published by the client.

```

higanbana@amberbox ~/git_project/mqtt_cybersec_project/pass (master)
$ python3 pub.py
Connected to MQTT Broker
Send 'temperature: -5 °C' to topic '<103825154>/temperature'
Send 'temperature: 52 °C' to topic '<103825154>/temperature'
Send 'temperature: 14 °C' to topic '<103825154>/temperature'
Send 'temperature: -9 °C' to topic '<103825154>/temperature'
Send 'temperature: 50 °C' to topic '<103825154>/temperature'
Send 'temperature: 61 °C' to topic '<103825154>/temperature'
Send 'temperature: 92 °C' to topic '<103825154>/temperature'
Send 'temperature: -3 °C' to topic '<103825154>/temperature'
Send 'temperature: 66 °C' to topic '<103825154>/temperature'
Send 'temperature: 49 °C' to topic '<103825154>/temperature'
Send 'temperature: 49 °C' to topic '<103825154>/temperature'
Send 'temperature: 50 °C' to topic '<103825154>/temperature'
Send 'temperature: -3 °C' to topic '<103825154>/temperature'
^CTerminate program
higanbana@amberbox ~/git_project/mqtt_cybersec_project/pass (master)
$

higanbana@amberbox ~/git_project/mqtt_cybersec_project/pass (master)
$ vim sub.py
higanbana@amberbox ~/git_project/mqtt_cybersec_project/pass (master)
$ python3 sub.py
Connected to mqtt broker
Received '{"topic": "<103825154>/temperature", "hash": "c9bee8176f319041b9e0d0fbce044568f715ae64fade64b5436f4fb0128c945e", "msg": "temperature: 23.01 Celsius"}' from '<103825154>/temperature' topic
received 'temperature: -5 °C' from '<103825154>/temperature' topic
received 'temperature: 52 °C' from '<103825154>/temperature' topic
received 'temperature: 14 °C' from '<103825154>/temperature' topic
received 'temperature: -9 °C' from '<103825154>/temperature' topic
received 'temperature: 50 °C' from '<103825154>/temperature' topic
received 'temperature: 61 °C' from '<103825154>/temperature' topic
received 'temperature: 92 °C' from '<103825154>/temperature' topic
received 'temperature: 32 °C' from '<103825154>/temperature' topic
received 'temperature: -3 °C' from '<103825154>/temperature' topic
received 'temperature: 66 °C' from '<103825154>/temperature' topic
received 'temperature: 49 °C' from '<103825154>/temperature' topic
received 'temperature: 49 °C' from '<103825154>/temperature' topic
received 'temperature: 50 °C' from '<103825154>/temperature' topic
received 'temperature: -3 °C' from '<103825154>/temperature' topic
^CTerminate program
higanbana@amberbox ~/git_project/mqtt_cybersec_project/pass (master)
$

```

Figure 1: Pass task demonstration

2.3 Credit Task

In the credit task, one client must both publish and subscribe to a topic and both clients subscribe to the public topic. To achieve this, I modify the publish client by calling subscribe() function before calling loop_start() to publish messages. The testing procedure is nearly the same as the Pass Task.

The current MQTT deployment is vulnerable to several security threats.

- **Confidentiality and Integrity vulnerability:** Messages are transmitted in plain text, making them susceptible to eavesdropping. Anyone with network access can intercept and capture messages using tools like Wireshark, compromising confidentiality and integrity. Public channels are not adequately secured, allowing unauthorized access and possible exposure of sensitive information. Client devices should exercise caution when handling messages received from public channels.
- **Bad authentication and authorisation:** Another significant issue is the use of weak authentication and authorization mechanisms. The same username and password are used for all clients, creating a single point of failure. If compromised, unauthorized access to the MQTT broker becomes possible. Furthermore, the lack of multifactor authentication weakens security as it relies solely on the possession of a username and password.

To address these cybersecurity issues, several measures should be implemented:

- **Encryption:** Transport Layer Security (TLS) should be used to protect messages from eavesdropping.
- **Limit access of public channel:** Secure public channels should be implemented to restrict access and ensure that only authorized clients can subscribe to them.
- **Client certificates and ACLs:** Authentication and authorization mechanisms should be strengthened by using client certificates to authenticate devices and implementing Access Control Lists (ACLs) to define permissions. Multifactor authentication can also be considered to improve security.

2.4 Distinction Task

This time, I need two devices that at least subscribe to different topics, respond to different requests, and at least one client can generate messages to multiple topics. Furthermore, the most interesting part of this challenge is to mimic the Graphical MQTT client (namely MQTTX) with a Python client. The best GUI library to simply create an application like this is Python Tkinter, which is a very minimal library with small footprints. Looking at the figure 2, The UI consists of four label frames: Connection frame, Subscribe frame, Publish frame, and Messages frame. The connection frame is responsible for broker connection containing broker server, port, username and password entry. Subscribe and Publish frame are pretty explanatory, they have fields for specifying topics and messages and they also have QoS combobox and Retain checkbox in addition. Last but not least, the Messages frame consists of a single scrolled text widget to log out user activities to see whether mqtt functionalities are working or not. In addition, the GUI program is capable of publish and subscribe to multiple topics by using comma separator for topic names.

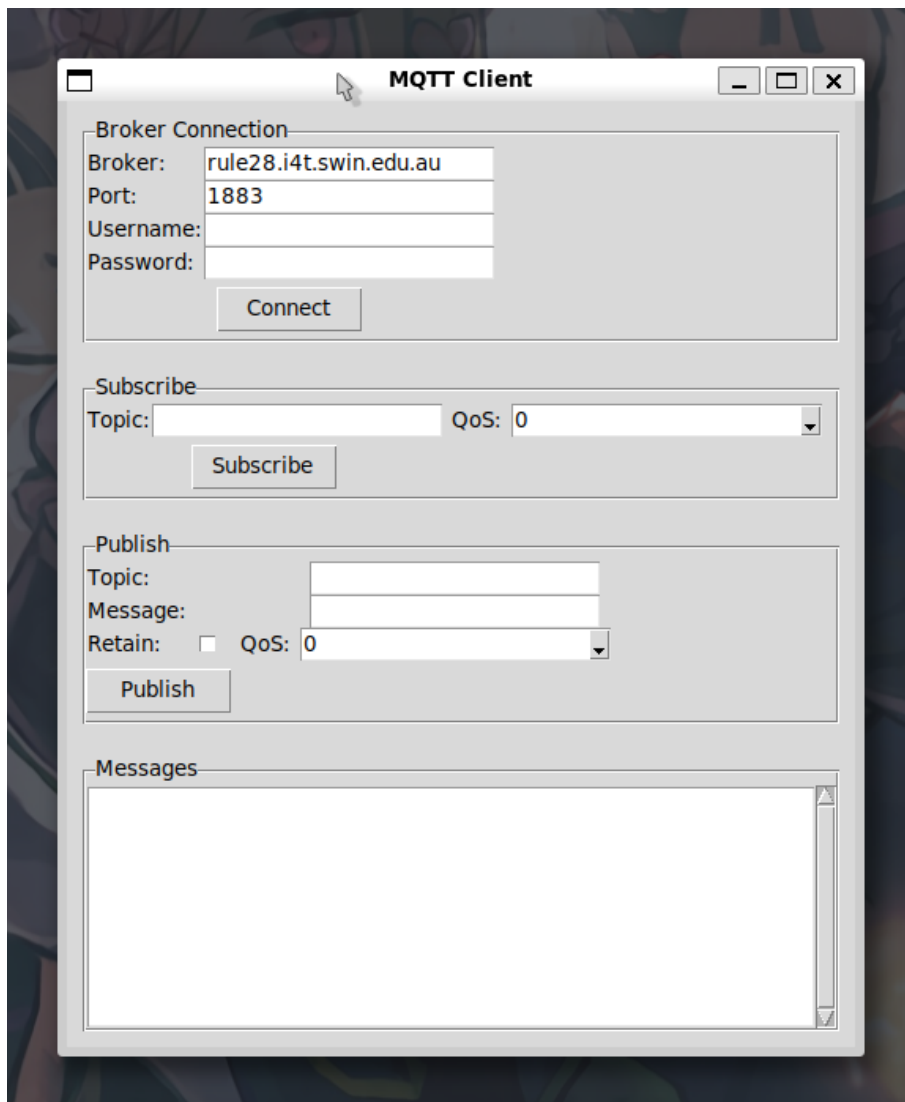


Figure 2: Distinction GUI Program

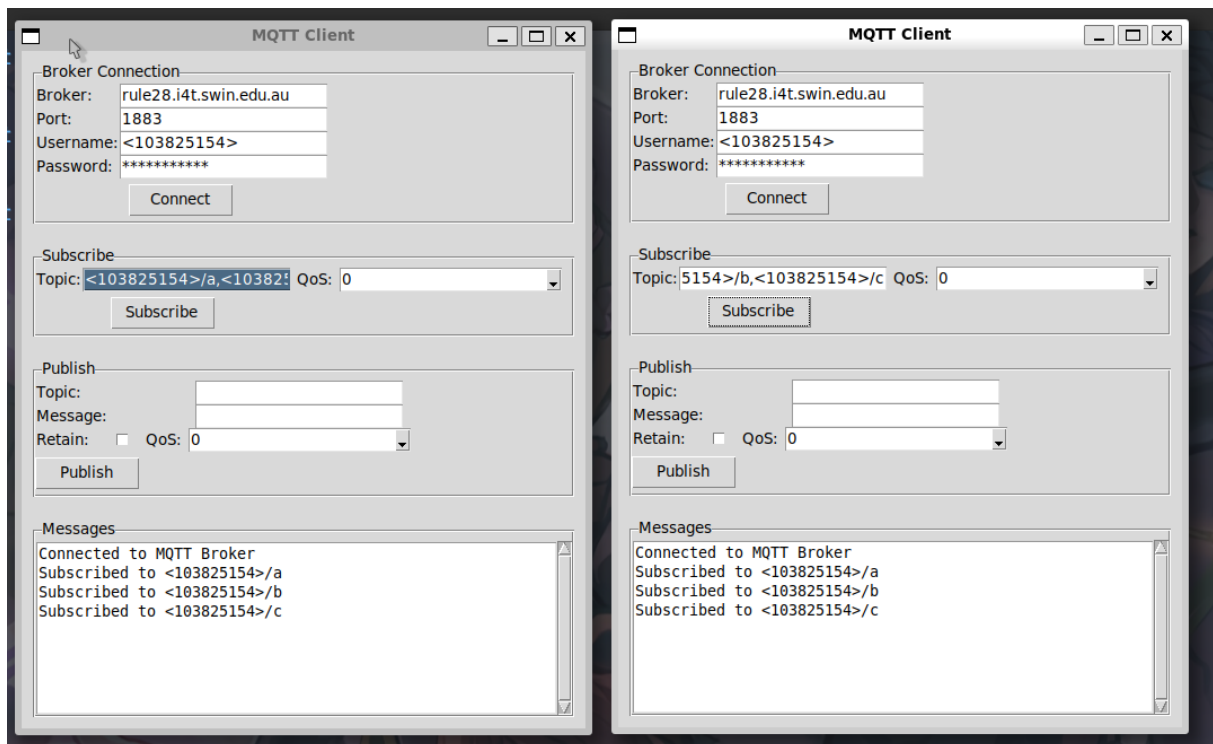


Figure 3: Two end devices subscribe to multiple different topics

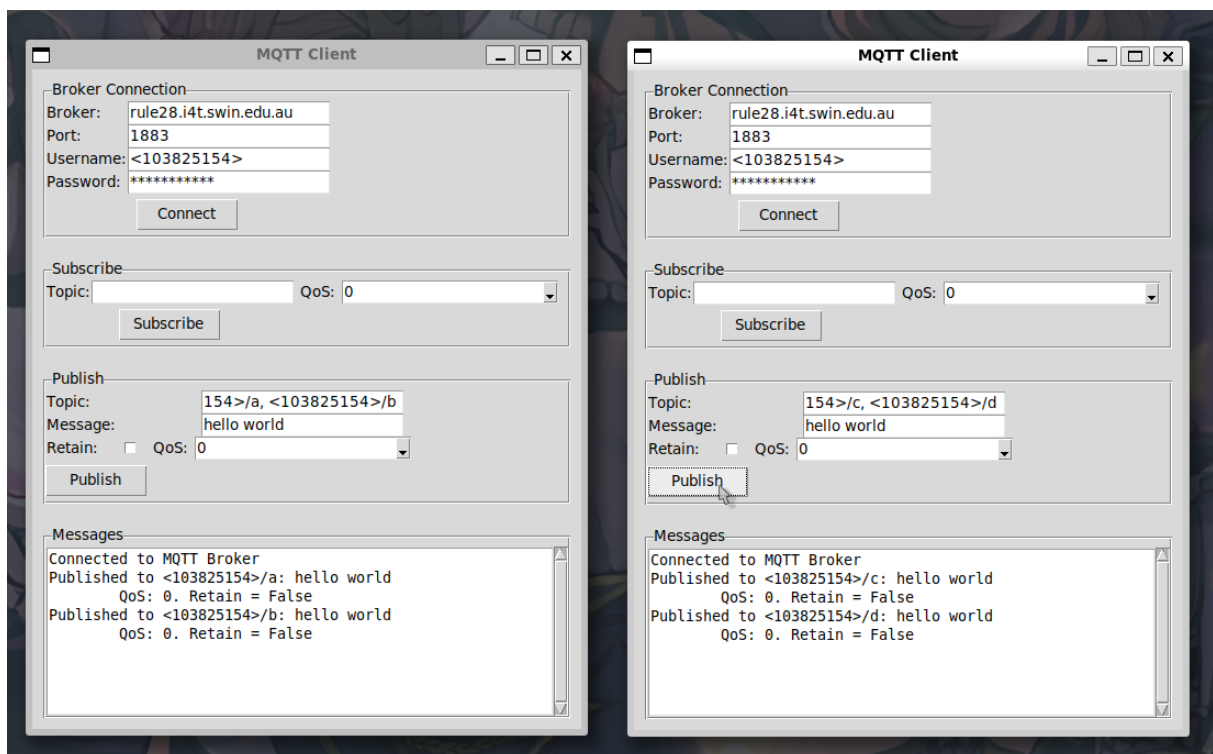


Figure 4: Two end devices publish messages to multiple different topics

2.5 High Distinction Task

This is when I finally start working on the project. The project will consist of two main components:

1. **Equipment Device:** A client that sends sensor readings and receives control commands
2. **Monitor Device:** A client that monitors sensor readings, generates alerts, and provides a GUI

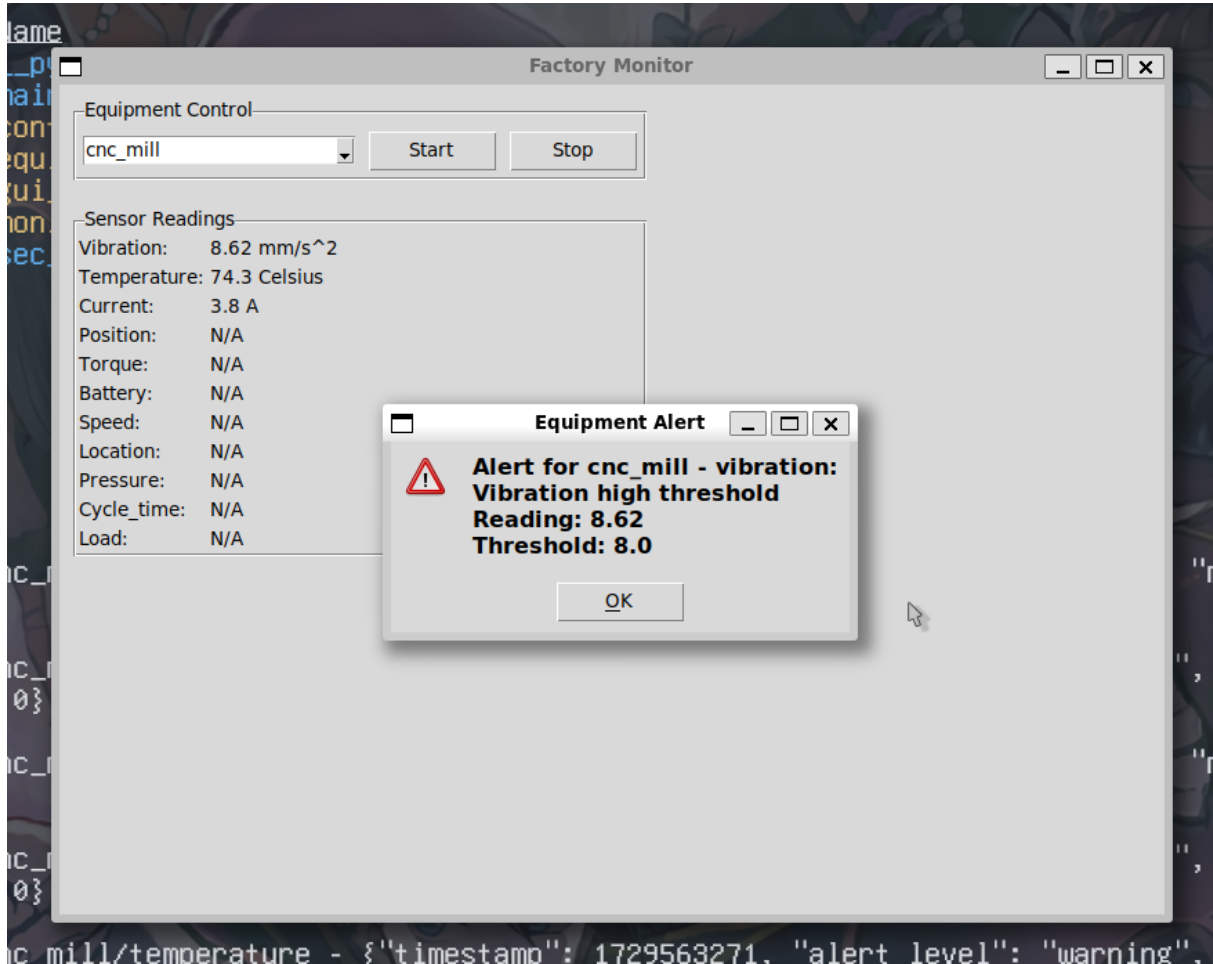


Figure 5: High distinction program

As for the equipment device, after establishing connection using appropriate credentials, it sends sensor readings to "factory/equipment/<equipment type>/<sensor type>" topic. It also subscribes to the "factory/command/" topic to receive instruction on when to turn sensors on or off for a specific equipment type; multiple equipments can run concurrently using Python's threading library. Then it logs maintenance requests to a file when there is an alert message from the monitor device. In terms of monitor device, it first subscribes to sensor reading topics, analyzes them in real time, and compares them against predefined thresholds. If sensor readings exceed thresholds, publish an alert message to the "factory/alert/<equipment type>" topic. Last but not least, I provided a GUI program that displays sensor readings, allows users to send start/stop commands, and shows alert messages in the message box.

Before deploying the project to the outside world, there are few majority concerns on cybersecurity issues:

- **Distributed Denial-of-Service (DDoS) attack:** Attackers can slow down the performance and latency of the server or eventually crash connection handles of

the entire network by spamming the public channels that devices are required to subscribe to. This can be resolved by setting a limit rate for sending messages, along with multiple security practices such as firewall, captcha, caching, and so on.

- **Man-in-the-Middle (MitM) attack:** this attack captures packets between the client and the broker and modifies the payload content of the message, causing different commands to be executed and different messages to be transmitted. This can be resolved by using end-to-end encryption for authorised parties, even if hackers can access to packets, they only see messages being ciphered, which can be only decrypted by using private keys from authorised identities.
- **Malicious payload exposure:** Malicious code can be inserted into unprotected payload, then they will be sent to connected devices and compromise them. Intrusion Detection System (IDS) is one of a viable solutions for this matter by keep track of harmful patterns and experiences from trusted vulnerabilities database